Soft Comput (2011) 15:1351–1371
DOI 10.1007/s00500-010-0675-y

ORIGINAL PAPER

Model accuracy in the Bayesian optimization algorithm

Claudio F. Lima · Fernando G. Lobo · Martin Pelikan · David E. Goldberg

Published online: 9 December 2010

Abstract Evolutionary algorithms (EAs) are particularly suited to solve problems for which there is not much information available. From this standpoint, estimation of distribution algorithms (EDAs), which guide the search by using probabilistic models of the population, have brought a new view to evolutionary computation. While solving a given problem with an EDA, the user has access to a set of models that reveal probabilistic dependencies between variables, an important source of information about the problem. However, as the complexity of the used models increases, the chance of overfitting and consequently reducing model interpretability, increases as well. This paper investigates the relationship between the probabilistic models learned by the Bayesian optimization algorithm (BOA) and the underlying problem structure. The purpose of the paper is threefold. First, model building in BOA is analyzed to understand how the problem structure is learned. Second, it is shown how the selection operator can lead to model overfitting in Bayesian EDAs. Third, the scoring metric that guides the search for an adequate model structure is modified to take into account the non-uniform distribution of the mating pool generated by tournament selection. Overall, this paper makes a contribution towards understanding and improving model accuracy in BOA, providing more interpretable models to assist efficiency enhancement techniques and human researchers.

Keywords Estimation of distribution algorithms · Bayesian optimization algorithm · Bayesian networks · Model overfitting · Selection

The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the Air Force Office of Scientific Research, the National Science Foundation, or the US Government.

C. F. Lima (✉)
Centre for Plant Integrative Biology, University of Nottingham, Sutton Bonington Campus, Loughborough LE12 5RD, UK
e-mail: clima.research@gmail.com

F. G. Lobo
Department of Electronics and Informatics Engineering, University of Algarve, Campus de Gambelas, 8000-117 Faro, Portugal
e-mail: fernando.lobo@gmail.com

M. Pelikan
Department of Mathematics and Computer Science, 320 CCB, University of Missouri at St. Louis, St. Louis, MO 63121, USA
e-mail: pelikan@cs.umsl.edu

D. E. Goldberg
Department of Industrial and Enterprise Systems Engineering, University of Illinois at Urbana-Champaign, Urbana, IL 61801, USA
e-mail: deg@illinois.edu

1 Introduction

The last decade has seen the rise and consolidation of a new trend of stochastic optimizers known as estimation of distribution algorithms (EDAs) (Pelikan et al. 2002; Larrañaga and Lozano 2002; Pelikan et al. 2006; Lozano et al. 2006). In essence, EDAs build probabilistic models of promising solutions and sample from the corresponding probability distributions to obtain new solutions. Therefore, these algorithms are typically classified according to the complexity of the probabilistic models they rely on. Simpler EDAs use a model of simple and fixed structure, and only learn the corresponding parameters. At the other side of the spectrum, there are EDAs with adaptive multivariate models

such as Bayesian networks (BNs) (Pearl 1988), which can model complex multivariate interactions. Examples of Bayesian EDAs include the Bayesian optimization algorithm (BOA) (Pelikan et al. 1999; Pelikan 2005), the estimation of Bayesian networks algorithm (EBNA) (Etxeberria and Larrañaga 1999), and the learning factorized distribution algorithm (LFDA) (Mühlenbein and Mahning 1999).

While Bayesian EDAs are able to solve a broad class of nearly decomposable and hierarchical problems in a reliable and scalable manner, their probabilistic models oftentimes do not exactly reflect the problem structure (Lima et al. 2007; Hauschild et al. 2009; Echegoyen et al. 2007; Mühlenbein 2008). Because these models are learned from a sample of limited size (population of individuals), particular features of the specific sample are also encoded, which act as noise when seeking for generalization. This is a well-known problem in machine learning, known as model overfitting. However, in the context of EDAs model overfitting is double-sided. While the goal is to model promising solutions rather than the entire search space, focusing on an excessively narrowed portion of this space might not reveal meaningful information about the underlying problem structure, and even reduce the probability of finding the optimum.

In many situations, the knowledge of the problem structure can be as valuable as a high-quality solution to the problem. This is the case for several model-based efficiency enhancement techniques (Sastry et al. 2004; Pelikan and Sastry 2004; Sastry et al. 2006; Sastry and Goldberg 2004; Lima et al. 2005, 2006, 2009; Lima 2009; Sastry et al. 2005; Yu and Goldberg 2004; Yu et al. 2007a; Hauschild et al. 2008; Hauschild and Pelikan 2008) developed for EDAs that yield *super-multiplicative speedups* (Goldberg and Sastry 2010). Another important situation is the *offline interpretation* of the probabilistic models (Yu et al. 2009; Yu and Goldberg 2004; Santana et al. 2008) to help develop fixed but structure-based operators for specific instances or classes of problems that have similar structure. In this case the EDA can act as a data miner to gain insight about the problem. The importance of analyzing the resulting probabilistic models in EDAs has also been recently highlighted by others (Santana et al. 2005; Wu and Shapiro 2006; Correa and Shapiro 2006; Echegoyen et al. 2007; Hauschild et al. 2009; Lima et al. 2007; Mühlenbein 2008).

Bayesian network learning is an active topic of research in machine learning, as the choice of the search procedure can have a great influence on model accuracy. However, the problem of finding the best network has been proven to be NP-complete for most scoring metrics (Chickering et al. 1994). Therefore, in Bayesian EDAs a simple local search procedure is typically used for a good compromise between search efficiency and model quality (Pelikan et al. 1999; Pelikan 2005; Etxeberria and Larrañaga 1999; Mühlenbein and Mahning 1999), given the high computational cost of considering more sophisticated alternatives (Echegoyen et al. 2007). Note that Bayesian networks (or any other probabilistic model for that matter) are used as an auxiliary tool in the optimization process, thus it is a good practice to keep the search complexity as simple as possible. On the other hand, this work focuses on the best way to integrate BNs within the evolutionary computation framework to improve the expressiveness of the learned models.

This paper investigates model structural accuracy in the Bayesian optimization algorithm, giving particular emphasis to the relationship between the underlying problem structure and the learned Bayesian network structure. The paper also addresses the selection operator as a source of overfitting in Bayesian EDAs. First, a detailed analysis of model learning in BOA is performed to better understand how the problem structure is learned and when inaccuracies are introduced in the network. Next, the role of selection in BN learning is investigated by looking at selection as the mating pool distribution generator, which turns out to have a great impact on model structural accuracy. Particularly, it is shown that tournament selection generates the mating pool according to a power distribution that leads to model overfitting. However, if the metric that scores networks takes into account the resampling bias induced by tournament selection, the model quality can be highly improved and comparable to that of truncation selection which generates a uniform distribution, more suitable for BN learning. The utility of the proposed scoring metric is verified through experiments on three test problems that represent different facets of probabilistic modeling in Bayesian EDAs. These problems are the *m–K* trap, the onemax, and the hierarchical trap. The results show that the model structural accuracy and interpretability is significantly improved with the modified scoring metric. In general, this paper contributes to better understand and interpret the probabilistic models in BOA, knowledge that can be used to improve several efficiency enhancement techniques (Sastry et al. 2004, 2005, 2006; Pelikan and Sastry 2004; Sastry and Goldberg 2004; Lima et al. 2005, 2006, 2009; Lima 2009; Yu and Goldberg 2004; Yu et al. 2007a, 2009; Hauschild et al. 2008; Hauschild and Pelikan 2008; Goldberg and Sastry 2010) and assist human researchers (Yu et al. 2009; Yu and Goldberg 2004).

The paper is organized as follows. The next section introduces relevant background to understand the purpose of the paper by describing BOA and previous work related to the topic addressed. Section 3 analyses in detail how the model is learned in BOA, while Sect. 4 investigates the role of selection in model learning and overfitting. Section 5 models the scoring metric gain when overfitting with tournament selection. In Sect. 6, an adaptive scoring metric is proposed to avoid overfitting, which is shown to considerably improve model accuracy. The paper ends with major conclusions.

2 Background

2.1 Bayesian optimization algorithm

The Bayesian optimization algorithm (BOA) (Pelikan et al. 1999; Pelikan 2005) uses Bayesian networks (BNs) to capture the (in)dependencies between the decision variables of the optimization problem. In BOA, the traditional crossover and mutation operators of evolutionary algorithms are replaced by (1) building a BN which models promising solutions and (2) sampling from the probability distribution encoded by the built BN to generate new solutions. The pseudocode of BOA is detailed in Fig. 1.

Bayesian networks (Pearl 1988) are powerful graphical models that combine probability theory with graph theory to encode probabilistic relationships between variables of interest. A BN is defined by its structure and corresponding parameters. The structure is represented by a directed acyclic graph (Cormen et al. 1990) where the nodes correspond to the variables of the problem and the edges correspond to conditional dependencies. The parameters are represented by the conditional probabilities for each variable given at any instance of the variables that this variable depends on. More formally, a Bayesian network encodes the following joint probability distribution,

$$p(X) = \prod_{i=1}^{l} p(X_i|\Pi_i), \tag{1}$$

where $X = (X_1, X_2, \ldots, X_l)$ is a vector with all variables of the problem, Π_i is the set of *parents* of X_i (nodes from which there exists an edge to X_i), and $p(X_i|\Pi_i)$ is the conditional probability of X_i given its parents Π_i.

The parameters of a Bayesian network can be represented by a set of conditional probability tables (CPTs) or local structures. Using local structures such as decision trees allows a more efficient and flexible representation of local conditional distributions, improving the expressiveness of BNs (Chickering et al. 1997; Friedman and Goldszmidt 1999; Pelikan 2005). In this work we focus on BNs with decision trees.

Bayesian optimization algorithm (BOA)

(1) Create a random population P of n individuals.
(2) Evaluate population P.
(3) Select P' individuals from P using a selection procedure.
(4) Model the selected individuals P' by learning the most adequate Bayesian network B.
(5) Create a new population O by sampling from the joint probability distribution of B.
(6) Evaluate population O.
(7) Replace all (or some) individuals in population P by those from O.
(8) If stopping criteria are not satisfied, return to step 3.

Fig. 1 Pseudocode of the Bayesian optimization algorithm

The quality of a given network structure is quantified by a scoring metric. We consider two popular metrics for BNs: the Bayesian–Dirichlet metric (BD) (Cooper and Herskovits 1992; Heckerman et al. 1994) and the Bayesian information criterion (BIC) (Schwarz 1978).

The BD metric for BNs with decision trees (Chickering et al. 1997) is given by

$$\begin{aligned}\mathrm{BD}(B) = \; & p(B) \prod_{i=1}^{l} \prod_{l \in L_i} \frac{\Gamma(m'_i(l))}{\Gamma(m_i(l) + m'_i(l))} \\ & \prod_{x_i} \frac{\Gamma(m_i(x_i, l) + m'_i(x_i, l))}{\Gamma(m'_i(x_i, l))},\end{aligned} \tag{2}$$

where $p(B)$ is the prior probability of the network structure B, L_i is the set of leaves in the decision tree T_i (corresponding to X_i), $m_i(l)$ is the number of instances in the population that contain the traversal path in T_i ending in leaf l, $m_i(x_i, l)$ is the number of instances in the population that have $X_i = x_i$ and contain the traversal path in T_i ending in leaf l, $m'_i(l)$ and $m'_i(x_i, l)$ represent prior knowledge about the values of $m_i(l)$ and $m_i(x_i, l)$. Here, we consider the K2 variant of the BD metric, which uses an uninformative prior that assigns $m'_i(x_i, l) = 1$.

To favor simpler networks over more complex ones, the prior probability of each network $p(B)$ can be adjusted according to its complexity, that is given by the description length of the parameters required by the network (Chickering et al. 1997; Friedman and Goldszmidt 1999). Based on this principle, the following penalty function was proposed for BOA (Pelikan 2005),

$$p(B) = 2^{-0.5 \log_2(n) \sum_{i=1}^{l} |L_i|}, \tag{3}$$

where n is the population size, and $|L_i|$ is the number of leaves in decision tree T_i.

The BIC metric is based on the minimum description length (MDL) principle (Rissanen 1978) and is given by

$$\begin{aligned}&\mathrm{BIC}(B) \\ &= \sum_{i=1}^{l} \left(\sum_{l \in L_i} \sum_{x_i} \left(m_i(x_i, l) \log_2 \frac{m_i(x_i, l)}{m_i(l)} \right) - |L_i| \frac{\log_2(n)}{2} \right).\end{aligned} \tag{4}$$

It has been shown that the behavior of these metrics is asymptotically equivalent; however, the results obtained with each metric can differ for particular domains, particularly in terms of sensitivity to noise. In the context of EDAs, when using CPTs to store the parameters, the BIC metric outperforms the K2 metric, but when using decision trees or graphs, the K2 metric has shown to be more robust (Pelikan 2005). We will confirm this observation in the remainder of the paper.

To learn the most adequate structure for the BN a greedy algorithm is usually used as a good compromise between search efficiency and model quality. We consider a simple learning algorithm that starts with an empty network and at each step performs the operation that improves the metric the most, until no further improvement is possible. The operator considered is the *split*, which splits a leaf on some variable and creates two new children on the leaf. Each time a split on X_j takes place in tree T_i, an edge from X_j to X_i is added to the network. For more details on learning BNs with local structures the reader is referred elsewhere (Chickering et al. 1997; Friedman and Goldszmidt 1999; Pelikan 2005).

The generation of new solutions is done by sampling from the learned Bayesian network using probabilistic logic samping (PLS) (Henrion 1988). Briefly, PLS consists in (1) computing an ancestral ordering of the nodes (where each node is preceded by its parents) and (2) generating the values for each variable according to the ancestral ordering and the conditional probabilities (Eq. 1).

The hierarchical BOA (hBOA) was later introduced by Pelikan and Goldberg (2001) and Pelikan (2005) and results from the combination of BNs with local structures with a simple yet powerful niching method to maintain diversity in the population, known as restricted tournament replacement (RTR) (Harik 1995). hBOA is able to solve hierarchical decomposable problems, in which the variable interactions are present at more than a single level.

2.2 Related work

Although the main feature of BOA and other EDAs is to perform efficient mixing of key substructures or building blocks (BBs), they also provide additional information about the problem being solved. The probabilistic model of the population, that represents (in)dependencies among decision variables, is an important source of information which can be exploited to enhance the performance of EDAs even more, or to assist the user in a better interpretation and understanding of the underlying structure of the problem. Examples of using structural information from the probabilistic model for another purpose beyond mixing include the design of structure-aware crossover operators (Yu et al. 2009), fitness estimation (Sastry et al. 2004, 2006; Pelikan and Sastry 2004), induction of global neighborhoods for mutation operators (Sastry and Goldberg 2004; Lima et al. 2006; Lima 2009), hybridization and adaptive time continuation (Lima et al. 2005, 2006, 2009; Lima 2009), substructural niching (Sastry et al. 2005), offline (Yu and Goldberg 2004) and online (Yu et al. 2007a) population size adaptation, and the speedup of the model building itself (Hauschild et al. 2008; Hauschild and Pelikan 2008). Therefore, it is important to understand under which conditions the structural accuracy of the probabilistic models in BOA and other structure-learning EDAs can be maximized. Recently, some studies have been done in this direction (Santana et al. 2005; Wu and Shapiro 2006; Correa and Shapiro 2006; Echegoyen et al. 2007; Hauschild et al. 2009; Mühlenbein 2008). In the remainder of this section we take a brief look at these works.

Santana, Larrañaga, and Lozano (Santana et al. 2005) analyzed the effect of selection on the arousal of bivariate interactions between single variables for random functions. They showed that for these functions, independence relationships not represented by the function structure are likely to appear in the probabilistic model. The authors also noted that even if the function structure plays an important role in the creation of dependencies, this role is mediated by selection (Santana et al. 2005). Additionally, an EDA that only used a subset of the dependencies that exist in the data (malign interactions) was proposed. Some preliminary experiments showed that these approximations of the probabilistic model can in certain cases be applied to EDAs.

Wu and Shapiro (2006) investigated the presence of overfitting when learning the probabilistic models in BOA and its consequences in terms of overall performance when solving random 3-SAT problems. CPTs (to encode the conditional probabilities) and the corresponding BIC metric were used. The authors concluded that overfitting does take place and that there is some correlation between this phenomenon and performance. The reduction in overfitting was proposed by using an early stopping criterion (based on cross entropy) for the learning process of BNs, which gave some improvement in performance.

The trade-off between model complexity and performance in BOA was also studied by Correa and Shapiro (2006). They looked at the performance achieved by BOA as a function of a parameter that determines the maximum number of incoming edges for each node. This parameter puts a limit on the number of parents for each variable, simplifying the search procedure for a model structure. This parameter was found to have a strong effect on the performance of the algorithm, for which there is a limited set of values where the performance can be maximized. These results were obtained using CPTs and the corresponding K2 metric. We should note that in fact this parameter is crucial if CPTs are used with the K2 metric; however, this is not the case for more sophisticated metrics that efficiently incorporate a complexity term to introduce pressure toward simpler models. This can be done better with the BIC metric for CPTs, or with the K2 metric for the case of decision trees (Pelikan 2005).

Echegoyen et al. (2007) applied new developments in exact BN learning into the EDA framework to analyze the consequent gains in optimization. While in terms of convergence time the gain was marginal, the models learned by EBNA were more closely related to the underlying structure of the problem. However, the computational cost of learning exact BNs is only manageable for relatively small problem sizes (experiments were made for a maximum problem size of 20).

Hauschild et al. (2009) made an empirical analysis of the probabilistic models built by hierarchical BOA for several test problems. The authors verified that the models learned closely correspond to the problem structure and do not change much over consequent iterations. They have also concluded that creating adequate probabilistic models for the 2D Ising spin glasses problem by hand is not straightforward even with complete knowledge of the problem. While in that work Hauschild et al. used truncation selection, this paper demonstrates that the results from Hauschild et al. (2009) do not carry over to other selection methods that assign several copies of the same individual to the mating pool according to a non-uniform distribution.

Recently, Muhlenbein (2008) investigated the Bayesian networks learned for LFDA and BOA when solving a trap-5 decomposable function. He found that in order to find the optimum about 80–90% of the edges have to be correctly identified. Also, the penalty factor used for the BIC metric was shown to have influence on the network density. Although these results are relevant to better understand Bayesian EDAs, there is a fundamental difference from our study—the maximum number of incoming edges was set according to the problem structure—which reduces dramatically the overfitting phenomenon. In real-world optimization this is not typically the case, therefore we let the algorithm learn by itself the adequate complexity. Another important difference is that both LFDA and BOA used CPTs to encode the model parameters, as opposed to DTs used in this work. Additionally, while LFDA used truncation selection, BOA was paired with tournament selection, resulting in worse model quality when compared to LFDA (Mühlenbein 2008). The author however did not make any remarks for the reason of such quality difference.

On the contrary, this work demonstrates that the difference in model quality is actually caused by the selection procedure rather than the algorithm as a whole. Furthermore, we show when and why overfitting is related to the selection method and propose a method to counterbalance this feature.

3 Analyzing model expressiveness

This section analyzes model learning in BOA when solving a problem of known structure and where that knowledge is crucial to solve it efficiently. We start by introducing the experimental setup used along the paper and then proceed to a detailed analysis of the learning process of BNs in BOA.

3.1 Experimental setup for measuring structural accuracy of probabilistic models

We start by clarifying some terms that are relevant to the scope of this paper.

Definition 1 The fitness function of an additively decomposable problem (ADP) is defined as:

$$f(X_1, X_2, \ldots, X_l) = \sum_{i=1}^{m} f_i(X_{S_i}), \tag{5}$$

where l is the number of decision variables, X_i is the ith variable, m is the number of subfunctions, f_i is the ith subfunction, and $S_i \in \{1, 2, \ldots, l\}$ is the subset of variable indexes for subfunction f_i.

Definition 2 Two problem variables are said to be *linked* if there is at least one subfunction f_i that is defined over both variables.

Definition 3 An *edge* (regardless of its direction) is *correct* if it connects two variables that are linked according to the fitness function definition.

Definition 4 The *model structural accuracy* (MSA) is defined as the ratio of correct edges over the total number of edges in the Bayesian network.

Definition 5 *Model overfitting* is defined as the inclusion of *incorrect (or unnecessary) edges* to the Bayesian network, which leads to excessive complexity.

To investigate the MSA in BOA, we focus on solving an ADP of known structure, where it is clear which dependencies must be discovered (for successful tractability) and which dependencies are unnecessary (reducing the interpretability of the models). The test problem considered is the m–k trap function:

$$f(X) = \sum_{i=1}^{m} f_{\text{trap}}(X_{S_i}), \tag{6}$$

where m is the number of concatenated k-bit trap functions and S_i is the index set of the variables belonging to the ith trap function. For example, with $k = 3$ and $m = 2$, the corresponding index sets are $I_1 = \{1, 2, 3\}$ and $I_2 = \{4, 5, 6\}$.

Trap functions (Ackley 1987; Deb and Goldberg 1993) are relevant to test problem design because they bound an important class of nearly decomposable problems (Goldberg and Sastry 2010). The trap function used (Deb and Goldberg 1993) is defined as follows

$$f_{\text{trap}}(u) = \begin{cases} k, & \text{if } u = k \\ k - 1 - u, & \text{otherwise} \end{cases} \quad (7)$$

where u is the number of ones in the string, k is the size of the trap function. Note that for $k \geq 3$ the trap function is fully deceptive (Deb and Goldberg 1993) which means that any lower than k-order statistics will mislead the search away from the optimum. In this problem the accurate identification and exchange of the building-blocks (BBs) is critical to achieve success, because processing substructures of lower order leads to exponential scalability (Thierens and Goldberg 1993; Thierens 1999). Note that no information about the problem is given to the algorithm; therefore, it is equally difficult for BOA if the variables correlated are closely or randomly distributed along the chromosome string.

Definition 6 A *clique* is a set of nodes N such that for every two nodes in N, there exist an edge (regardless its direction) connecting them.

If k variables interact with each other, the probabilistic model should express their joint distribution to be able to maintain k-order statistics. For example, the joint distribution of variables X_1, X_2, X_3, and X_4 can be expressed in a Bayesian network as

$$p(X_1, X_2, X_3, X_4) = p(X_1)p(X_2|X_1)p(X_3|X_1, X_2)\, p(X_4|X_1, X_2, X_3), \quad (8)$$

or any other permutation of the variables that respect this dependency structure. Figure 2 shows the ideal BN for the m–k trap problem with $k = 4$. As can be observed, the ideal Bayesian network structure should contain a clique between interacting variables, where the order of the edges is defined in such way that there are no cycles (so that sampling new instances with PLS is feasible). Additionally, the model should not contain dependencies between different trap subfunctions.

Later, in Sect. 6 we extend our experiments to onemax and hierarchical trap functions to validate our approach on different sources of difficulty in model learning with Bayesian EDAs. These functions will be detailed later in that section.

For all experiments, we use the minimal population size required to solve the problem in 10 out of 10 independent runs (success rate of 100%). The population size is obtained by performing 10 independent bisection runs (Sastry 2001; Pelikan 2005). Therefore, the total number of function evaluations is averaged over 100 (10 × 10) runs. To focus on the influence of selection in model quality, the replacement strategy is kept as simple as possible, where the offspring fully replace the parent population.

3.2 A closer look at model learning

When analyzing the dependencies captured by the Bayesian networks in BOA, it can be observed that while all important linkages are detected—given a sufficient population size (Pelikan et al. 2003; Pelikan 2005; Yu et al. 2007b)—spurious dependencies are also incorporated in the model. Although the structure of the BN captures such excessive complexity, the corresponding conditional probabilities oftentimes nearly express independence between spurious variables and correct linkage groups, therefore not affecting the capability of sampling such variables as if they were almost independent. However, as model overfitting increases with problem size and selection pressure (Lima et al. 2007), the interpretation of the resulting models becomes meaningless.

To better understand the excessive complexity of the learned models in BOA, we make a detailed analysis at the model learning process. Figure 3 shows the scoring metric gain obtained in model building for a single run of BOA. For each learning step (edge addition) the corresponding gain in the scoring metric is plotted, as well as if the edge inserted is correct (upper dots) or spurious (lower dots). The first, middle, and last generations of the run are plotted using binary tournament selection. Looking at the results, one can easily conclude that the K2 metric is better for learning the underlying problem structure because it

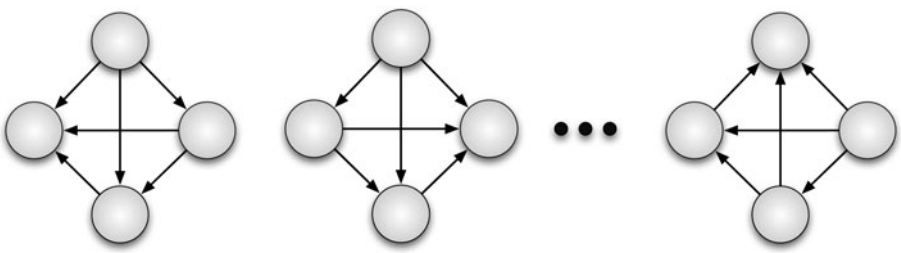


Fig. 2 Ideal Bayesian network to solve the m–k trap problem with $k = 4$. A clique is formed for each set of variables corresponding to a trap function, while between different traps there are no dependencies

introduces much less spurious dependencies than the BIC metric. Nevertheless, some incorrect edges are also inserted in the network for the K2 metric, particularly at the end of the run.

The number of correct edges (n_{ce}) in a Bayesian network for the m–k trap problem is given by

$$n_{ce} = \sum_{i=1}^{m} \frac{k_i(k_i - 1)}{2}, \text{ for } k \geq 2, \quad (9)$$

where m is the number of subfunctions and k_i is the size (number of interacting variables) of the ith subfunction. Therefore, for the concatenated trap problem with $k = 5$ and $m = 10$ there is a total of 100 correct edges. While for the K2 metric at most 20 incorrect edges are inserted in the network, for the BIC metric a maximum of 350 incorrect edges (generation 6) are learned! Nevertheless, the problem is solved by BOA, although the model building phase with BIC takes more time. For the K2 metric, it can also be seen that while only 90% of the correct edges have been learned, the problem could still be solved (recall however that this refers to a single run). This result agrees with the observation made elsewhere (Mühlenbein 2008).

Analyzing BOA with the more robust K2 metric, we can see that the metric gain is higher at the beginning of the learning process and decreases towards zero; which is the threshold for accepting a modification in the Bayesian network. Additionally, the metric gain magnitude increases towards the end of the run (note that the maximum metric gain at generation 1 is 81, while at generation 11 is 8,913). This is due to the fact that as the search focus on specific regions of the search space (loss in diversity) the marginal likelihood of the model increases. Indeed, for the last generation the shape of the metric gain function is less noisy when compared to the first and middle generations. With respect to the correctness of the edges, we can observe that the overwhelming majority of the spurious edges are inserted in the network at the end of the learning procedure. This suggests that an earlier stopping criterion or a higher acceptance threshold (note that spurious edges have a metric gain that is quite small) in the learning procedure could avoid the acceptance of a great part of incorrect edges.

Before discussing the role of selection in model quality, we elaborate on the particular shape of the metric gain function observed for the last generation in Fig. 3 (more clear for the K2 metric). This function appears to have the shape of several decreasing steps, where the metric gain drops considerably at some points in BN learning. These are indeed different stages of dependency learning. Figure 4 shows an example with the different stages in

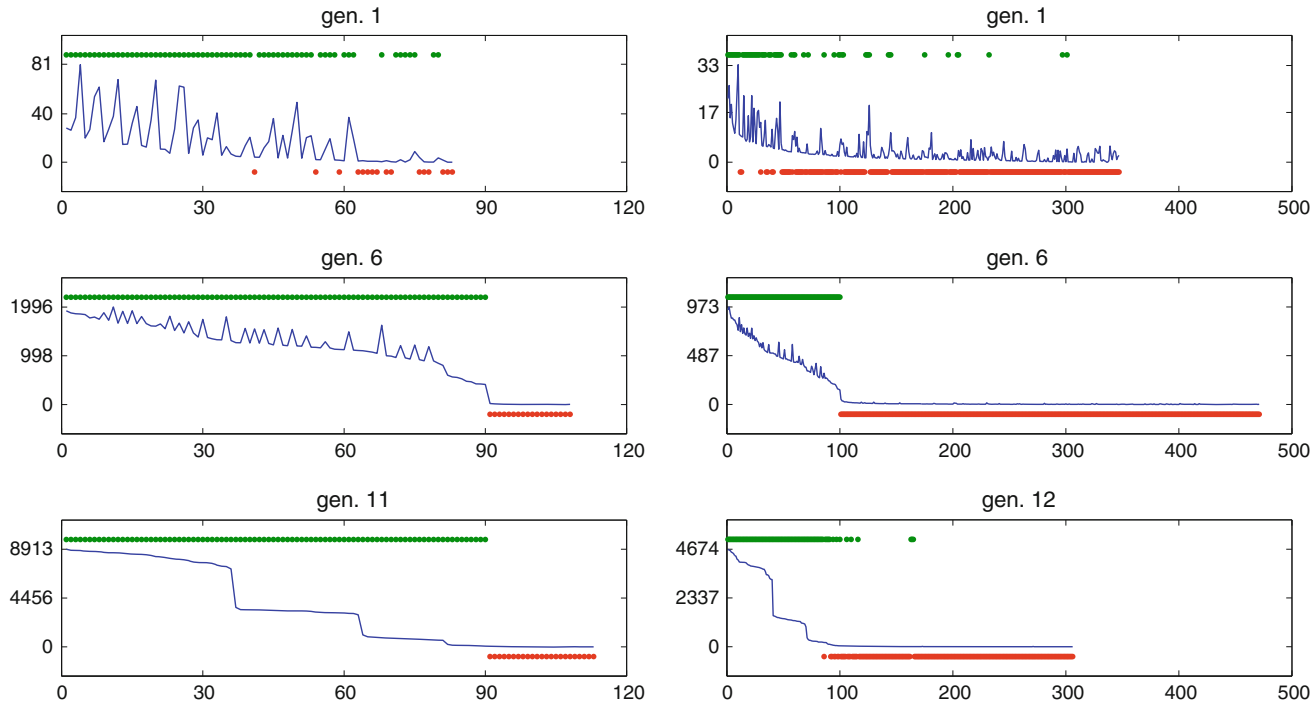


Fig. 3 Metric gain versus learning step in Bayesian network learning for a single run with BOA using the K2 (*left*) and BIC (*right*) metrics. The *upper dots* are correct edge additions while the *lower* ones are incorrect edges. The problem is the 5-bit trap with $m = 10$ ($l = 50$) for which there is a total of 100 correct edges. The first, middle, and last generations of the run are plotted. Binary tournament selection and minimal population size required (to solve the problem) is used. Clearly, the K2 metric produces more accurate models than the BIC metric, when using BNs with local structures

Bayesian network learning when solving a 5-bit trap function. Four different stages can be identified, where the order in which edges are added to the network is determined by their corresponding metric gain. Remember that the greedy algorithm for learning the network structure accepts at each step the edge that improves the scoring metric the most. Note also that the metric gain corresponding to adding an edge from a given node X_a to another node X_b is inversely proportional to the number of parent nodes that X_b already contains (Ahn and Ramakrishna 2008). This can also be concluded from the population size requirements for adding a correct edge in BOA (Pelikan et al. 2003; Pelikan 2005), which scales as $O(2^{\mu}l^{1.05})$, where μ is the number of parents nodes that X_b already contains. Therefore, the magnitude of the metric gain for the $k-1$ different stages will decrease towards later stages.

Looking at the metric gain function for generation 11 with the K2 metric (Fig. 3), the conjecture pictured in Fig. 4 can be confirmed. For $m = 10$ concatenated trap functions of $k = 5$, there should be $10 \times 4 = 40$ edges for the first stage, $10 \times 3 = 30$ for the second stage, $10 \times 2 = 20$ edges for the third stage, and $10 \times 1 = 10$ edges for the fourth and last stage of BN learning. These estimated stages closely match the observed shape for the metric gain function. Note that some of the edges might not be learned (about 10% in this case) or learned at a different stage from the expected one, due to noise coming from learning with a finite population size; however, the above estimates are still quite accurate. To confirm our rationale, we plot in Fig. 5 the last generation of a run with BOA + K2 when solving the same problem with different number of subfunctions $(k = 5, m = 20)$ and different trap size $k = 4$, $m = 10$. Once again, the metric gain decreases more or less at the expected points where there is a stage shift.

Typically, incorrect edges are added to the network at the latter stages of learning, when the metric gain is smaller. This means that model overfitting comes mainly in the form of excessive number of incoming parents for the nodes that depend on some other interacting variables. For example, in stage 4 (Fig. 4d), when learning the dependency $X_1 \rightarrow X_5$, additional spurious dependencies are oftentimes added to the network, such as $X_6 \rightarrow X_5$, $X_7 \rightarrow X_5$, etc.

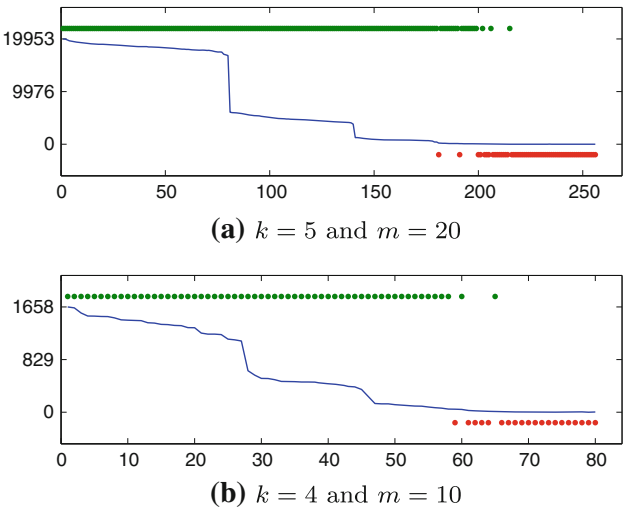


Fig. 5 Metric gain versus learning step in Bayesian network learning for a single run with BOA using the K2 metric. The last generation is plotted for a trap problem with **a** $k = 5$, $m = 20$ and **b** $k = 4$, $m = 10$ which have a total of 200 and 60 correct edges, respectively. The decrease on metric gain closely match the different stages of BN learning

4 The role of selection

When EDAs model the set of promising solutions to guide the search, these are identified by a selection method, which might have a great influence on their performance (Harik et al. 1999; Johnson and Shapiro 2001; Santana

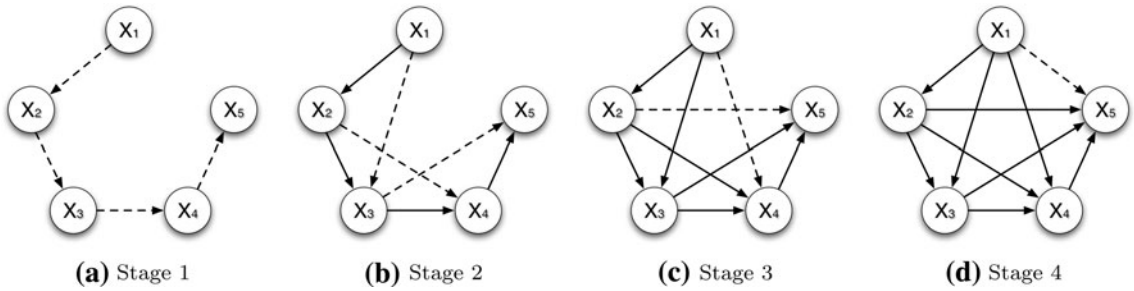


Fig. 4 Different stages of Bayesian network learning in BOA when solving a trap function with $k = 5$. The order in which dependencies are learned is determined by their corresponding metric gain. Remember that the greedy algorithm for learning the network structure accepts at each step the edge that improves the scoring metric the most. Also note that the metric gain corresponding to adding an edge from a node X_a to another node X_b is inversely proportional to the number of parent nodes that X_b already contains (Ahn and Ramakrishna 2008). Therefore, the magnitude of the metric gain for the $k-1$ different stages will differ. Typically, incorrect edges are added to the network at the latter stages of learning, when the metric gain is smaller

et al. 2005; Lima et al. 2007; Yu et al. 2007b). This section pays special attention to the role of selection in BOA. In particular, we consider two widely used selection schemes in EDAs: tournament and truncation selection.

In tournament selection (Goldberg et al. 1989; Brindle 1981), s individuals are randomly picked from the population and the best one is selected for the mating pool. This process is repeated n times, where n is the population size. There are two popular variations of tournament selection, with and without replacement. With replacement, the individuals are drawn from the population following a discrete uniform distribution. Without replacement, individuals are also drawn randomly from the population but it is guaranteed that every individual participates in exactly s tournaments. While the expected outcome for both alternatives is the same, the latter is a less noisy process. Therefore, in this study we use tournament selection without replacement.

In truncation selection (Mühlenbein and Schlierkamp-Voosen 1993) the best $\tau\%$ individuals in the population are selected for the mating pool. This method is equivalent to the standard (μ, λ)-selection procedure used in evolution strategies (ESs), where $\tau = \frac{\mu}{\lambda} \times 100$.

Note that when increasing the size of the tournament s, or decreasing the threshold τ, the selection intensity increases. In order to compare the two selection operators on a fair basis, different configurations for both methods with equivalent selection intensity are considered. The relation between selection intensity I, tournament size s, and truncation threshold τ is taken from (Blickle and Thiele 1997) and is shown in Table 1.

The influence of the selection strategy in BOA has been discussed in detail elsewhere (Lima et al. 2007). Here, essential findings for the purpose of studying model overfitting are reviewed and extended to the BIC metric. Figure 6 shows the model quality and number of function evaluations for different combinations of selection methods and scoring metrics. From a model quality perspective, it is clear that (1) truncation selection performs better than tournament selection and (2) K2 metric performs better than BIC metric. Note that with tournament selection, while for small values of s the number of evaluations decreases, after some value of s, the number of evaluations starts to increase again. Curiously, this happens when the MSA approaches 0.1.

Table 1 Equivalent tournament size (s) and truncation threshold (τ) for the same selection intensity (I)

I	0.56	0.84	1.03	1.16	1.35	1.54	1.87
s	2	3	4	5	7	10	20
$\tau(\%)$	66	47	36	30	22	15	8

4.1 Selection as the mating pool distribution generator

Like in traditional genetics, the selection mechanism is responsible for ensuring the survival of the fittest in the population. In the context of EDAs, this is one of the most important components inherited from the evolutionary computation framework. However, in EDAs, which have a strong connection with data mining and classification, the selection operator can also be viewed as the generator of the data set used to learn the probabilistic model at each generation. Since in EDAs we are interested in modeling the set of promising solutions, the selection operator indicates which individuals have relevant features to be modeled and propagated in the solution set. Before moving to the study of the selection strategy as the data set generator for learning the BNs, we make a simple analysis of the selection operators considered.

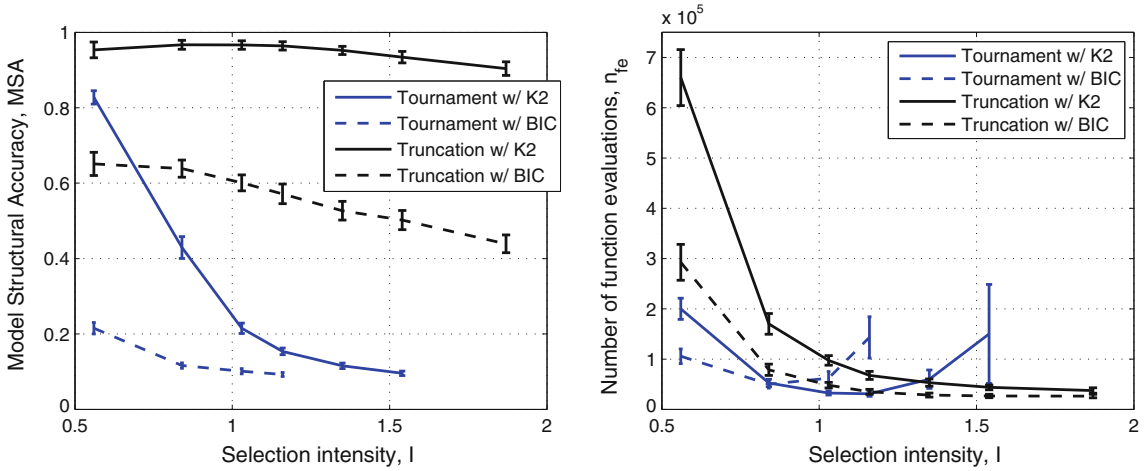
In terms of creating duplicate individuals in the population there are two responsible mechanisms. The selection operator explicitly assigns several copies of the same individual to the mating pool, where the number of copies is somewhat proportional to their fitness rank. This is the case for tournament, ranking, and proportional selection. Additionally, the model sampling procedure generates with a certain probability duplicates of the same individual, although selection implicitly controls how often this happens. Note that this probability will increase in time as the EDA starts focusing on more concrete regions of the search space. Clearly, the selection operator has some influence on this phenomenon as it explicitly regulates the convergence speed of the algorithm. Without loss of generality, consider that the replication of individuals done explicitly by the selection operator is the main source of duplicates in the population.

For the sake of simplicity, let us assume that all individuals have different fitness. Ordering the population by fitness, where the worst individual has rank 1 and the best has rank n, the probability that an individual with rank i wins a given tournament of size s is, for $i \geq s$, given by

$$p_i = \frac{\binom{i-1}{s-1}}{\binom{n-1}{s-1}} = \frac{(i-1)!(n-s)!}{(i-s)!(n-1)!} = \prod_{j=1}^{s-1} \frac{i-j}{n-j}, \text{ for } s > 2. \tag{10}$$

Note that the worst $s-1$ individuals will never win a tournament because the same individual cannot be selected more than once for the same tournament when using tournament selection without replacement. Therefore for $i < s$, $p_i = 0$.

Given that in tournament selection without replacement each individual participates in exactly s tournaments, the expected number of copies (c_i) in the mating pool for an individual of rank i is simply



(a) Model structural accuracy **(b)** Num. of function evaluations

Fig. 6 Model structural accuracy and number of function evaluations for different selection–metric combinations when solving the 5-bit trap problem of size $l = 50$

$$c_i = sp_i. \tag{11}$$

For $i \gg s$, and consequently $n \gg s$, the distribution of the expected number of copies c_i can be approximated by a power distribution (Balakrishnan and Nevzorov 2003) with p.d.f.,

$$f(x) = \alpha x^{\alpha-1},\ 0<x<1,\ \alpha = s. \tag{12}$$

In this way, the distribution of c_i can be expressed for any population size, where the relative rank is given by $x = i/n$. Note that as the relative rank slightly decreases from 1 the corresponding number of expected copies rapidly decreases. This is particularly true for higher tournament sizes, when increasing the exponent of the power factor.

On the other hand, in truncation selection the expected number of copies for the selected individuals is one, which follows a uniform distribution with p.d.f.,

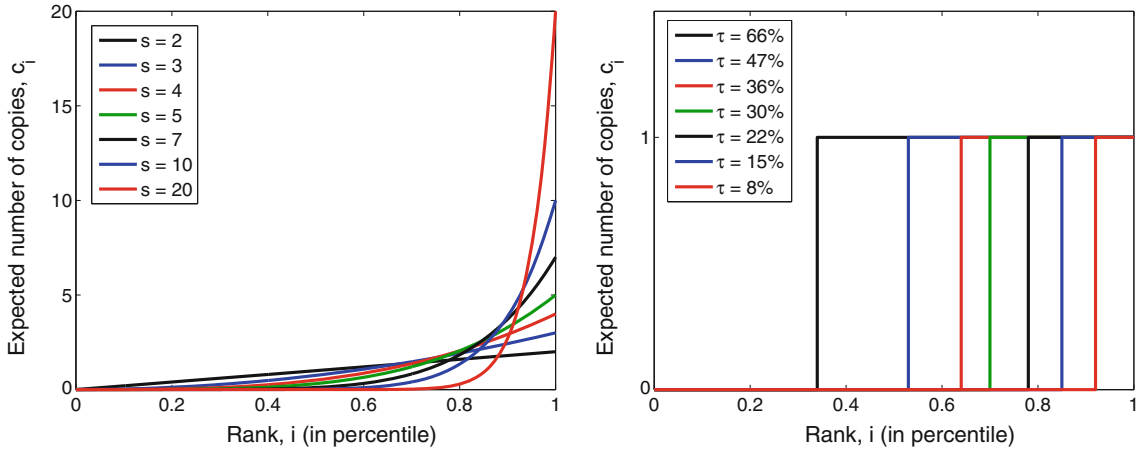
$$c_i = \begin{cases} 0, & \text{if } i<n\,(1-(\tau/100)) \\ 1, & \text{otherwise.} \end{cases} \tag{13}$$

Figure 7 shows the distributions of expected number of copies for each individual with rank expressed in percentile. The difference between the two selection methods is notorious. While tournament selection assigns increasing relevance to top-ranked individuals according to a power distribution, truncation selection gives no particular preference to any of the selected individuals, all having the same frequency in the learning data set.

The differences between tournament and truncation distributions stress out two relevant features of any given selection method: (1) *window size*, which determines the proportion of unique individuals that are included in the mating pool, and (2) *distribution shape*, which determines the relevance of each selected individual in the mating pool, in terms of the number of copies. These features in a certain way control the tradeoff between exploration and exploitation in model structural learning in EDAs.

Clearly, tournament and truncation selection differ in both features. While the window size is deterministically defined in truncation selection—solutions above the threshold are included in the selected set and solutions below are not—in tournament selection, the choice of which individuals to include in the mating pool is a stochastic process (except for the best solution and the worst $s - 1$), but also guided by fitness rank. The probability of inclusion rapidly decreases with rank, particularly for larger tournament sizes, as can be seen in Fig. 7a. In terms of distribution shape, the two selection methods also differ significantly. Tournament selection gives higher emphasis to top-ranked solutions according to a power distribution with $\alpha = s$. This means that best solutions get approximately s copies in the mating pool, which forces the learned models to focus on particular features of these individuals, which contain good substructures, but also undesirable components.

Another way to look at tournament selection in comparison with truncation selection as the mating pool generator is recognizing that this selection procedure acts as a biased data resampling on a uniform data set. The uniform data set is the set of unique selected solutions (solutions that will win at least one tournament), similar to what happens in truncation, while the resampling is performed when top-ranked individuals participate in more than one tournament. This sort of resampling is clearly biased by fitness.



(a) Tournament **(b)** Truncation

Fig. 7 Distribution of the expected number of copies in the mating pool for **a** tournament and **b** truncation selection with different selection intensity values. Note that s and τ values generate the same selection intensity. Rank is expressed in percentile

A related topic in data mining is the generalization of features with low-density in the learning data set, generally known as learning from imbalanced data sets (Japkowicz and Stephen 2002; Kubat and Matwin 1997; Weiss and Provost 2003; Pyle 1999). One way to achieve this is to artificially create additional data instances that appear to have the feature of interest, but in a way that has little impact on the distribution of the population as a whole. The last remark however can be quite difficult to ensure, as the question of what is the "natural" distribution of the data does not have a straightforward answer. Quoting from Pyle (1999):

> When added to the original data set, these now appear as more instances with the feature, increasing the apparent count and increasing the feature density in the overall data set. The added density means that mining tools will generalize their predictions from the multiplied data set. A problem is that any noise or bias present will be multiplied too.

This is indeed the problem with tournament selection when trying to model the overall problem structure.

5 Modeling metric gain when overfitting

This section establishes a relationship between the scoring metric values when overfitting and the tournament size. As a first step, we want to compute how the power-law-based frequencies differ from uniform ones in the mating pool.

To analyze the effect of tournament size on resampling bias we must look at the cumulative distribution function (c.d.f.) of the power distribution, which is given by

$$F(x) = x^s, 0 < x < 1, s \geq 1. \quad (14)$$

Note that for $s = 1$ we have a uniform distribution and there's no resampling, as the mating pool becomes a complete copy of the population. For $s \geq 2$, we can obtain the proportion of individuals in the mating pool with rank equal or less than x by simply calculating $F(x)$, or alternatively, the market share of the $(1 - x)$ top-ranked individuals given by $1 - F(x)$ (corresponding to the right-side area of the c.d.f.).

The overfitting due to noise coming from top-ranked individuals is certainly more likely to happen if we think in a fairly small percentage of the population. Said differently, the smaller this proportion is, the more likely these individuals will contain the same misleading features that are induced by noise. On the other hand, this proportion should be significant enough in terms of relative frequencies so that it can influence the metric component that scores the likelihood of the model with respect to the data. How large or small should this proportion be depends obviously on the tournament size. For larger tournament sizes, this proportion is expected to be inferior to the case of smaller tournament sizes because the number of copies assigned to top individuals increases considerably. Therefore, we recognize that this proportion should be small, but the exact proportion will differ from situation to situation.

To better illustrate our argument, Fig. 8 shows the power c.d.f. for several proportions of top-ranked individuals. It can be seen that for small proportions ($\leq$1) of top-ranked individuals, the expected proportion in the mating pool after selection grows approximately linearly with the tournament size. Note that as the proportion considered is more elitist, the slope of the linear relationship approaches the proportion itself. For example, when considering the best 0.1%, the market share after selection with $s = 50$ is $4.88\% \approx 0.1\% \times 50$.

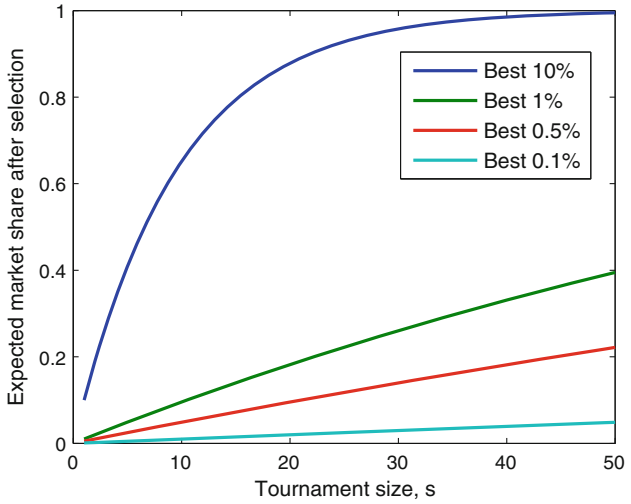


Fig. 8 Expected market share of top-ranked individuals included in the mating pool after selection for infinite population size. For small proportions (≤ 1) of top-ranked individuals the relation between tournament size and the expected proportion in the mating pool after selection is approximately linear

The bottom line of this rationale is to verify that, in the worst case, the noise in terms of counts or relative frequencies coming from the replication of top-ranked individuals grows linearly with the tournament size.

Consider now the possibility of adding an edge from a variable X_2 to another variable X_1 due to nonlinearities introduced by tournament selection, knowing that these two variables are in fact independent from each other. To investigate the influence of the resampling done by successive tournaments, we must derive the score metric for the network where an edge is added from X_2 to X_1. Given that both MDL and Bayesian metrics are decomposable, it is sufficient to look at the term corresponding to the node X_1. The metric gain G_{metric} obtained by splitting a leaf on X_2 in the tree encoding the parameters of X_1 and adding the corresponding edge to the network is given by

$$G_{\text{metric}} = \mathit{ScoreAfter} - \mathit{ScoreBefore} - \mathit{ComplexityPenalty}, \tag{15}$$

where *ScoreAfter* is the metric score obtained after splitting the leaf into two new ones, *ScoreBefore* is the score obtained before the split (keeping X_1 independent from X_2), and *ComplexityPenalty* is the penalty associated with the increased complexity of adding one leaf to the tree. In BOA, if this gain is positive the split is accepted and the corresponding edge is inserted in the Bayesian network.

Due to its simplicity (compared with the K2 metric), the BIC metric is considered in the following calculations. The metric gain corresponding to adding an edge from X_2 to X_1 is

$$\begin{aligned} G_{\text{BIC}} = {} & m(X_1X_2 = 00)\log_2\left(\frac{m(X_1X_2 = 00)}{m(X_2 = 0)}\right) \\ & + m(X_1X_2 = 10)\log_2\left(\frac{m(X_1X_2 = 10)}{m(X_2 = 0)}\right) \\ & + m(X_1X_2 = 01)\log_2\left(\frac{m(X_1X_2 = 01)}{m(X_2 = 1)}\right) \\ & + m(X_1X_2 = 11)\log_2\left(\frac{m(X_1X_2 = 11)}{m(X_2 = 1)}\right) \\ & - m(X_1 = 0)\log_2\left(\frac{m(X_1 = 0)}{n}\right) \\ & - m(X_1 = 1)\log_2\left(\frac{m(X_1 = 1)}{n}\right) \\ & - \frac{1}{2}\log_2(n), \end{aligned} \tag{16}$$

where $m(X_1X_2 = x_1x_2)$ is the number of individuals in the population with $X_1X_2 = x_1x_2$ and n is the population size. Note that the first four terms correspond to *ScoreAfter*, the fifth and sixth terms express *ScoreBefore*, and the final term penalizes the score because of the complexity added to the network. Denoting $m(X_1X_2 = x_1x_2)$ by $m_{x_1x_2}$ and recognizing that $m(X_1 = x_1) = m(X_1X_2 = x_10) + m(X_1X_2 = x_11)$, as well as $n = m_{00} + m_{01} + m_{10} + m_{11}$, the previous expression can be expressed as

$$\begin{aligned} G_{\text{BIC}} = {} & m_{00}\log_2\left(\frac{m_{00}}{m_{00} + m_{10}}\right) \\ & + m_{10}\log_2\left(\frac{m_{10}}{m_{00} + m_{10}}\right) \\ & + m_{01}\log_2\left(\frac{m_{01}}{m_{01} + m_{11}}\right) \\ & + m_{11}\log_2\left(\frac{m_{11}}{m_{01} + m_{11}}\right) \\ & - (m_{00} + m_{01})\log_2\left(\frac{m_{00} + m_{01}}{m_{00} + m_{01} + m_{10} + m_{11}}\right) \\ & - (m_{10} + m_{11})\log_2\left(\frac{m_{10} + m_{11}}{m_{00} + m_{01} + m_{10} + m_{11}}\right) \\ & - \frac{1}{2}\log_2(m_{00} + m_{01} + m_{10} + m_{11}). \end{aligned} \tag{17}$$

Expressing in terms of relative frequencies, the gain can be expressed as

$$G_{\text{BIC}} = nG'_{\text{BIC}} - \frac{1}{2}\log_2(n), \tag{18}$$

where

$$G'_{\mathrm{BIC}} = p_{00}\log_2\left(\frac{p_{00}}{p_{00}+p_{10}}\right) + p_{10}\log_2\left(\frac{p_{10}}{p_{00}+p_{10}}\right) + p_{01}\log_2\left(\frac{p_{01}}{p_{01}+p_{11}}\right) + p_{11}\log_2\left(\frac{p_{11}}{p_{01}+p_{11}}\right) - (p_{00}+p_{01})\log_2(p_{00}+p_{01}) - (p_{10}+p_{11})\log_2(p_{10}+p_{11}). \quad (19)$$

Next, we want to model the deviation from the actual frequencies in a uniformly distributed mating pool (in terms of copies) to biased frequencies towards the noise induced by the replication of top-ranked individuals (power distribution). First, consider the frequencies on the uniform mating pool to be $p_{00} = p_{01} = p_{10} = p_{11} = 0.25$, which reveals independence between X_1 and X_2. Then, and without loss of generality, we will assume that these frequencies are deviated towards equally increasing p_{00}, p_{11} and equally decreasing p_{01}, p_{10}. This assumption relies on the fact that the decrease in entropy (corresponding to an increase in score) will be achieved faster than for other possible configurations of pairwise frequency deviation. In this way, we analyze the case that can upper bound other possible deviations.

Assuming that the deviation of the "true" frequencies is linear with respect to the tournament size, as argued before, the frequency deviation can be expressed as

$$\begin{aligned} p_{00} &\approx 0.25 + \Delta(s-1), \\ p_{01} &\approx 0.25 - \Delta(s-1), \\ p_{10} &\approx 0.25 - \Delta(s-1), \\ p_{11} &\approx 0.25 + \Delta(s-1), \end{aligned} \quad (20)$$

where Δ is the slope of the linear relationship plotted in Fig. 8, therefore the exact value will depend on the proportion considered. Replacing (Eq. 20) into (Eq. 19) and denoting $(s-1)$ by s',

$$G'_{\mathrm{BIC}} \approx (0.25+\Delta s')\log_2\left(\frac{0.25+\Delta s'}{0.5}\right) + (0.25-\Delta s')\log_2\left(\frac{0.25-\Delta s'}{0.5}\right) + (0.25-\Delta s')\log_2\left(\frac{0.25-\Delta s'}{0.5}\right) + (0.25+\Delta s')\log_2\left(\frac{0.25+\Delta s'}{0.5}\right) - 0.5\log_2(0.5) - 0.5\log_2(0.5). \quad (21)$$

Simplifying the previous equation, we have

$$G'_{\mathrm{BIC}} \approx (0.5+2\Delta s')\log_2\left(\frac{0.25+\Delta s'}{0.5}\right) + (0.5-2\Delta s')\log_2\left(\frac{0.25-\Delta s'}{0.5}\right) + 1. \quad (22)$$

Using the logarithm property $\log(a/b) = \log(a) - \log(b)$ and simplifying again, we get

$$G'_{\mathrm{BIC}} \approx (0.5+2\Delta s')\log_2(0.25+\Delta s') + (0.5-2\Delta s')\log_2(0.25-\Delta s') + 2. \quad (23)$$

Dividing both terms by 2,

$$\frac{1}{2}G'_{\mathrm{BIC}} \approx (0.25+\Delta s')\log_2(0.25+\Delta s') + (0.25-\Delta s')\log_2(0.25-\Delta s') + 1. \quad (24)$$

Looking at the function $x\log_2(x)$ for the interval $[0, 0.5]$, one can see that the first term in Eq. 24 is relatively constant around -0.5. Therefore,

$$\frac{1}{2}G'_{\mathrm{BIC}} \approx (0.25-\Delta s')\log_2(0.25-\Delta s') + 0.5, \quad (25)$$

or alternatively,

$$G'_{\mathrm{BIC}} \approx 2(0.25-\Delta s')\log_2(0.25-\Delta s') + 1. \quad (26)$$

The approximate expression for the metric gain G'_{BIC} due to overfitting of top-ranked individuals in tournament selection is plotted in Fig. 9. A value of $\Delta = 0.001$ is used (best 0.1%).

Since the schema proportions considered will vary from 0.25 to 0 or 0.5, the Δ value will basically define the increment/decrement step of those same proportions. For example, for a higher $\Delta = 0.005$ the approximate expression would be defined only for $s = [1, 50]$, instead of the plotted $s = [1, 250]$.

As can be seen, the metric gain grows close to linear in log-log scale, with the exception made for lower and higher values of s. This means a polynomial growth in linear scale, somewhere between linear and quadratic, which can be confirmed by comparison with reference curves. While

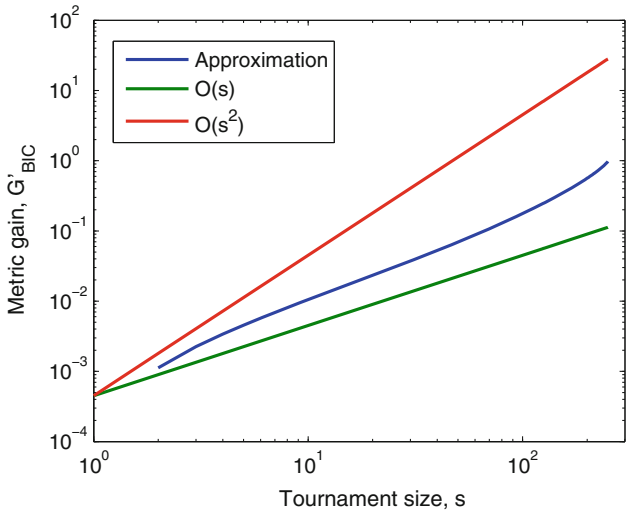


Fig. 9 Approximated metric gain G'_{BIC} due to overfitting of top-ranked individuals in tournament selection. A value of $\Delta = 0.001$ is used (best 0.1%). The growth of the metric gain is somewhere between linear and quadratic, but closer to linear

the metric gain G'_{BIC} does not account for the factor n (population size) and the complexity penalty term $0.5\log_2(n)$, it does tell us about the way the gain grows with respect to the tournament size s.

6 Improving model accuracy

In this section we change the complexity penalty of the scoring metric in order to account for the power distribution of tournament selection. First, we investigate the efficacy of different penalty factors that are functions of the tournament size s. Second, we validate the penalty found to be the most appropriate on two additional problems that pose new sources of difficulty to model learning in Bayesian EDAs.

6.1 Adaptive scoring metric

While the metrics considered have different backgrounds, the penalty associated with each leaf addition is exactly the same: $0.5\log_2(n)$ (Pelikan 2005). This becomes clear if we compare the logarithm of the K2 metric with the BIC metric. Additionally, the K2 metric has some implicit "penalty" for complex models, which comes from the marginal likelihood function. However, this is insufficient on its own for making models simple enough in Bayesian EDAs (both with standard CPTs and local structures). In fact, this extra penalty seems to be the reason for the K2 metric producing less complicated models than the BIC metric.

To compensate for the resampling bias of tournament selection, we aggravate the complexity penalty by a factor c_s that depends on the tournament size s, using $0.5c_s\log_2(n)$ instead of the standard penalty. In this way, the greater the number of copies of top-ranked individuals in the mating pool, the more demanding is the scoring function in accepting an edge/leaf addition. From the previous section we know that the metric gain due to overfitting grows approximately as $\Theta(s)$, therefore different c_s values around that estimate are tested to investigate the corresponding response in terms of MSA and number of function evaluations. In particular, we perform experiments for $c_s = \{\sqrt{s}, s, s\log_2(s)\}$ and compare them with the original penalty correction ($c_s = 1$).

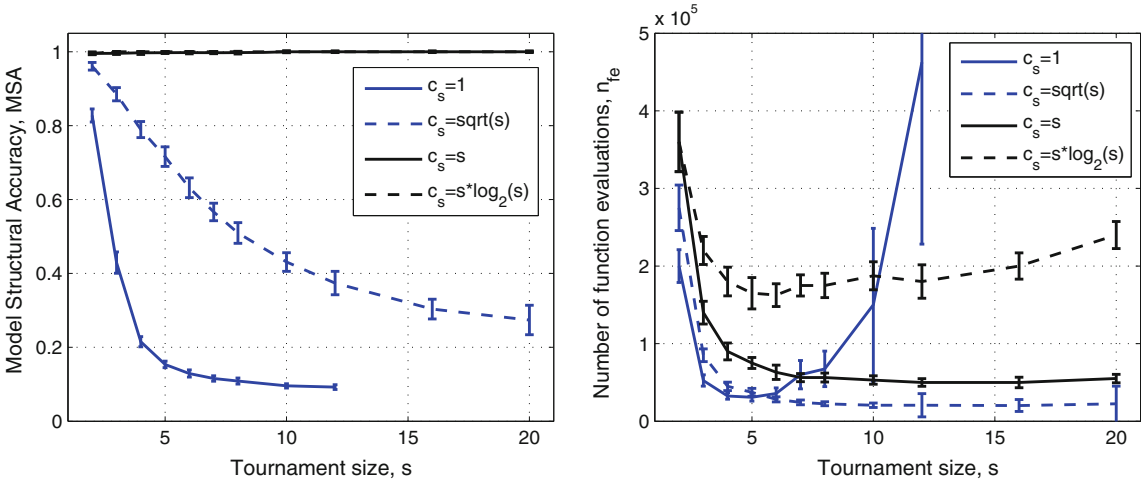
Experiments for both BIC and K2 metrics were performed, although BIC was shown to produce excessively complex models (both for tournament and truncation selection). Figures 10 and 11 (for K2 and BIC metrics, respectively) show the model quality and corresponding evaluations for BOA with tournament selection using different complexity penalties. Despite the difference observed between metrics, their behavior when increasing the complexity penalty is similar. Already for $c_s = \sqrt{s}$, the model quality improves with respect to the standard case $c_s = 1$, but when considering $c_s = s$ and $c_s = s\log_2(s)$ the improvement is much better. Increasing the penalty by a factor of s or higher takes the model structural accuracy very close to 100%. However looking at the number of evaluations spent by each penalty, it is clear that $c_s = s\log_2(s)$ is too strong as a penalty because for larger s values it takes too many evaluations and the situation gets worse with increasing s. On the other hand, the $s-$penalty ($c_s = s$) shows to be an adequate penalty because while obtaining high-quality models the number of evaluations is kept constant after some tournament size. This point us out to another advantage of the $s-$penalty, because it allows to have a wider range of s values for which BOA performs well and at a relatively low cost.

We now look at the behavior of tournament selection with the $s-$penalty for different problem sizes and compare it to truncation selection with the standard penalty. Figures 12 and 13 show BOA using the K2 metric with tournament and truncation selection, respectively. Clearly, tournament selection with the $s-$penalty obtains better model quality than truncation selection with the standard penalty. Notice, however, that model quality is now plotted between 90 and 100%, because both methods obtain models of much better quality than tournament selection with the standard penalty. In terms of number of evaluations, tournament selection is still less expensive than truncation selection, but as selection intensity increases their costs become comparable.

Figure 14 shows BOA with tournament selection but now using the BIC metric and the $s-$penalty. The results for truncation selection with the standard penalty are not plotted as their quality is much inferior to those with the $s-$penalty, with MSA typically varying between 40–70% (see Fig. 6). For this metric, the $s-$penalty also improves dramatically the model quality obtained with the standard penalty; however, the results are still not as good as for the K2 metric. For higher selection pressures the model quality is close to 100%, but as tournament size decreases the MSA degrades, a tendency that gets stronger with increasing problem size. This confirms the tendency of the BIC metric to generate more dense networks.

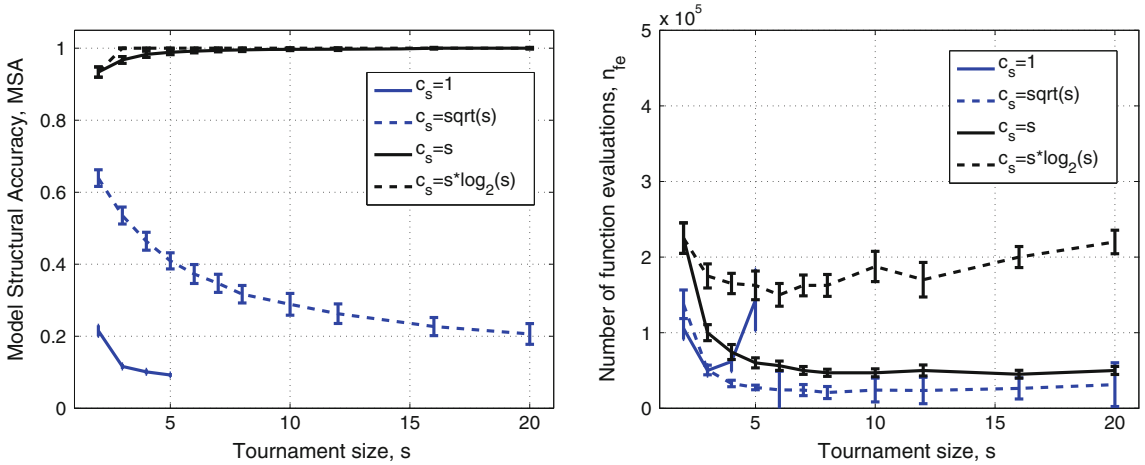
6.2 Method validation

While the previous results showed a significant improvement of model quality with the $s-$penalty, it is important to validate the proposed methodology when solving problems that pose new challenges in model learning for BOA and other Bayesian EDAs. Specifically, we consider the one-max and the hierarchical trap problems.



(a) Model structural accuracy **(b)** Num. of function evaluations

Fig. 10 Model quality and number of function evaluations for different penalty correction values $c_s = \{1, \sqrt{s}, s, s\log_2(s)\}$ with the K2 metric, when solving the trap problem with $k = 5$ and $m = 10$



(a) Model structural accuracy **(b)** Num. of function evaluations

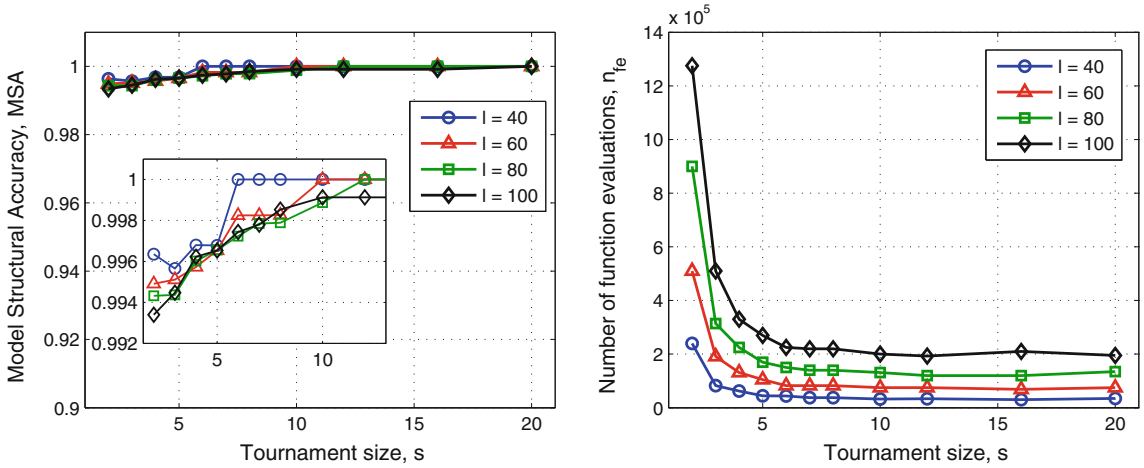
Fig. 11 Model quality and number of function evaluations for different selection-dependent values $c_s = \{1, \sqrt{s}, s, s\log_2(s)\}$ with the BIC metric, when solving the trap problem with $k = 5$ and $m = 10$

The onemax function (also known as counting ones) is simply defined as the sum of ones in a binary string. This is a simple linear function with the optimum in the solution with all ones. Therefore, there is no need of linkage information to be able to solve this problem. While the optimization of onemax is quite easy for univariate EDAs, the probabilistic models built by multivariate EDAs are known to easily introduce unnecessary dependencies which lead to increased population-sizing requirements.

On the other hand, the hierarchical trap problem (Pelikan and Goldberg 2001) poses a more difficult challenge to EDAs, as important dependencies are expressed at more than a single level. For these problems, the interactions in an upper level are too weak to be detected unless all lower levels are already solved. The hierarchical trap is composed by three components:

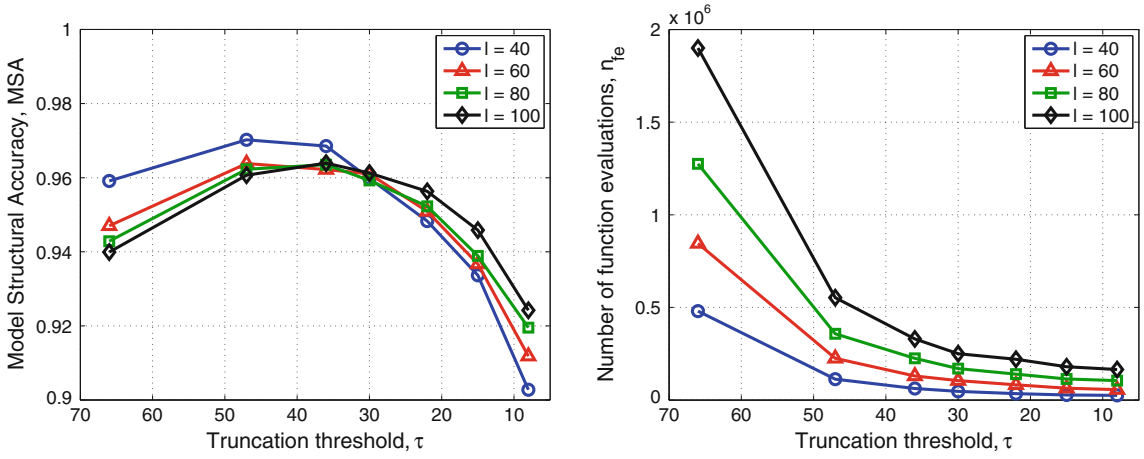
1. *Structure* which is defined by a balanced $k-$ary tree.
2. *Mapping function* that maps variables from a lower level to the next level. A block of all 0s and 1s is mapped to 0 and 1 respectively, and everything else is mapped to *null*.
3. *Contribution functions* are based on trap functions of order k. If any position of the string is *null*, the corresponding fitness contribution is zero.

Here we consider a 27-bit hierarchical trap with three levels and $k = 3$. See Fig. 15 for details. The contributions on each level are multiplied by third level so that the total



(a) Model structural accuracy **(b)** Num. of function evaluations

Fig. 12 Model quality and number of function evaluations for tournament selection with the K2 metric and $s-$penalty ($c_s = s$). The problem is the same 5-bit trap with different problem sizes $l = \{40, 60, 80, 100\}$

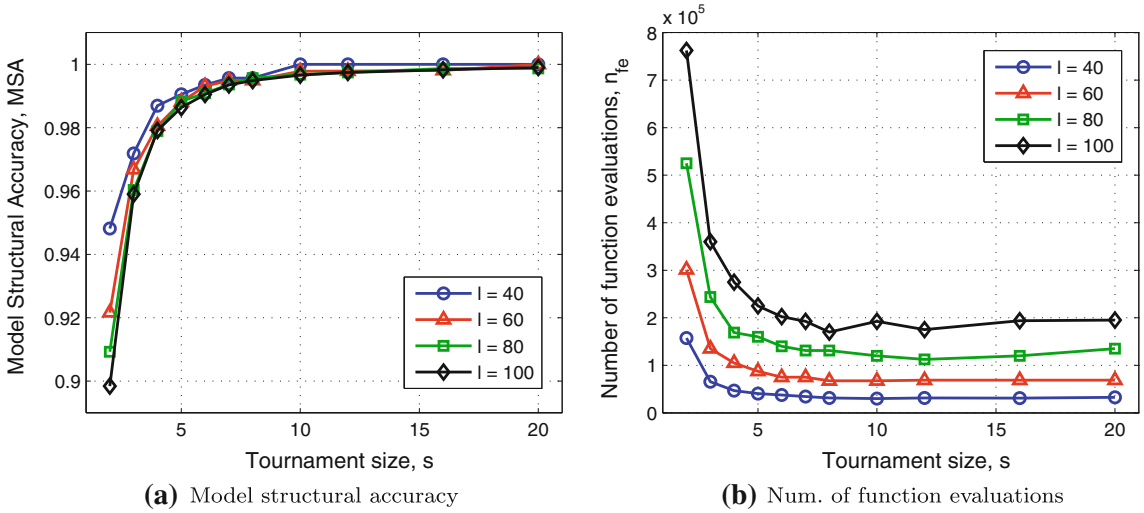


(a) Model structural accuracy **(b)** Num. of function evaluations

Fig. 13 Model quality and number of function evaluations for truncation selection with the K2 metric. The problem is the same 5-bit trap with different problem sizes $l = \{40, 60, 80, 100\}$

contribution of each level is the same. The optimal solution is given by the string with all 1s. Note that subsolutions 000 and 111 are equally good except for the top level; however, the local optimum 000 is easier to climb. Because we cannot distinguish between the two alternatives we must maintain them both until a decision can be reached at the top level. In order to be able to deal with this extra level of difficulty, niching methods are required to preserve diversity in the population without comprising evolutionary pressure towards better solutions. Therefore, for this problem we employ the hierarchical version of BOA by using restricted tournament replacement (Harik 1995; Pelikan 2005) to insert the offspring into the original population. Note that for hierarchical problems the probabilistic model must be capable of representing *chunks* of solutions from lower levels in a compact way so that only relevant features are considered (Pelikan 2005).

For the onemax problem, we have performed experiments for a string length of $l = 50$ and recorded the number of edges present in the Bayesian network. Note that the ideal model should not contain any edges because the variables are independent from each other. Therefore, the fewer edges the more accurate is the model. Figure 16 compares the model quality of the standard penalty with the $s-$penalty, along the run and for different tournament sizes. The plots are shown from a different angle to better observe the results. The number of edges in the networks for the standard penalty is considerably higher than those observed for the proposed penalty. Furthermore, when increasing selection



(a) Model structural accuracy **(b)** Num. of function evaluations

Fig. 14 Model quality and number of function evaluations for tournament selection with the BIC metric and $s-$penalty ($c_s = s$). The problem is the same 5-bit trap with different problem sizes $l = \{40, 60, 80, 100\}$

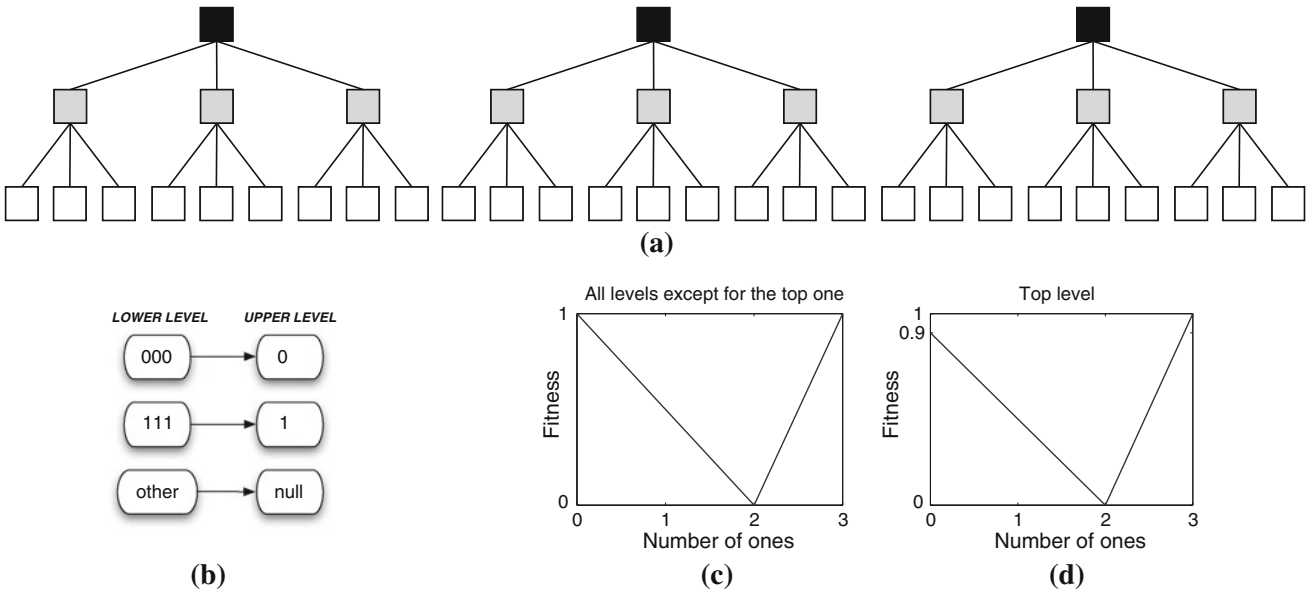


Fig. 15 The 27-bit hierarchical trap problem with three levels and $k = 3$. This problem is defined by three components: **a** structure, **b** mapping function, and **c**, **d** contribution functions. Note that 000 and 111 are equally good except for the top level. However, the local optimum 000 is easier to climb, which requires the maintenance of all alternatives until a decision can be reached at the top level

pressure for the standard penalty the resulting models become considerably more complex, which requires exponentially increasing population sizes with respect to tournament size s (to be able to solve the problem). This seems to reveal some sensitivity of BOA with tournament selection and the standard penalty in correctly modeling linear or approximately linear functions. On the contrary, when using the $s-$penalty the number of edges is drastically reduced, while the population-sizing requirements are considerably smaller, scaling approximately as $\Theta(s)$.

For the hierarchical trap problem the definition of correct/incorrect edges is not so clear. While the dependencies at the lower level are clearly correct and necessary dependencies, it is not clear when the dependencies at the higher levels should be captured. That will depend on whether the problem is already solved at the lower level. Therefore, instead of looking at the MSA, we explicitly plot the bivariate statistics relative to the dependencies learned by the Bayesian network.

Figure 17 plots the proportion of pairwise dependencies between variables for the 27-bit hierarchical trap

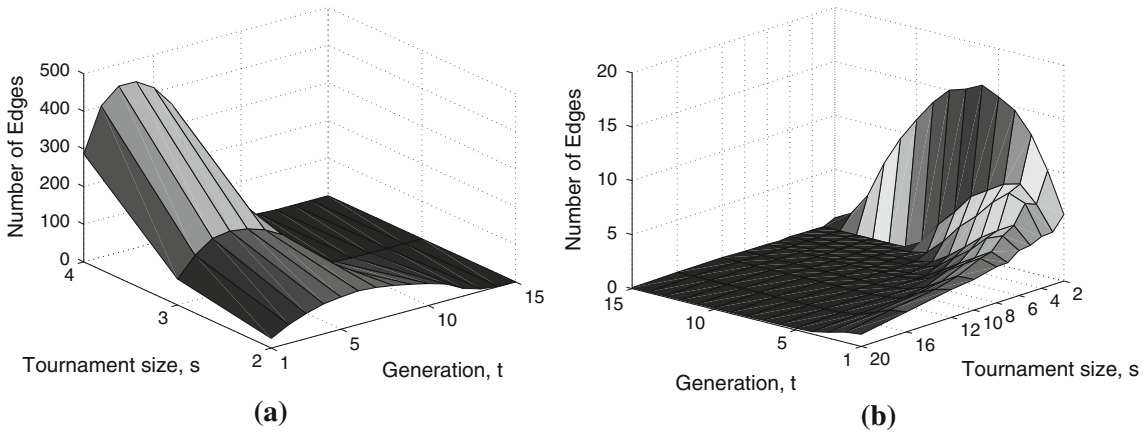


Fig. 16 Number of edges in the Bayesian network when solving the onemax problem with $l = 50$. Note that the ideal BN should not contain edges as the problem variables can be optimized independently from each other. The results for the **a** standard penalty and the **b** $s-$penalty are shown from a different angle so that they can be better observed. The $s-$penalty considerably reduces the number of edges captured by the network

captured by the hierarchical BOA in the last generation (when the optimum is found). The dependency proportion between any two variables refers to edges in both directions, which means that the graph is symmetrical with respect to the diagonal between points $(0, 0)$ and $(27, 27)$. The z-axis indicates the proportion of runs (out of 100 runs) in which a certain dependency was learned by the Bayesian network. For example, a dependency proportion of 0.87 between X_1 and X_2 means that in 87 out of 100 runs there is a dependency $X_1 \rightarrow X_2$ or $X_2 \rightarrow X_1$. Clearly, the graph obtained for the $s-$penalty is more informative about the underlying structure of the hierarchical problem than the one obtained for the standard penalty. Note that stronger dependencies (nine linkage groups of order $k = 3$) reveal the base level structure, while more weak dependencies (three linkage groups of order $k = 9$) denote the structure of the middle level. Because the problem is solved at the higher level, the optimum is found before the model can capture the next level structure.

With respect to truncation selection using the standard penalty, the conclusions are similar to those made for the *m–k* trap problem. Truncation selection achieves comparable (although marginally inferior) model quality to tournament with the $s-$penalty. In terms of function evaluations requirements, tournament selection is still less expensive than truncation, but for higher selection intensities their cost become similar.

Together, these results demonstrate that tournament selection is an efficient selection method for Bayesian EDAs, as it is for genetic algorithms, as long as the complexity penalty of the scoring metric takes into account the power distribution in the mating pool. The greater the tournament size is, the more demanding the scoring metric should be in accepting edge/leaf additions.

7 Summary and conclusions

While the main goal of the Bayesian optimization and other Bayesian estimation of distribution algorithms is to perform efficient mixing of partial solutions, it also provides additional information about the problem. The Bayesian networks learned during the run, which represent probabilistic dependencies between variables, are an important source of information that can be exploited to improve performance, or to get a better insight on the problem itself. Thus, it is important to investigate the relationship between the learned probabilistic models and the underlying problem structure.

Since the main objective of EDAs as stochastic optimizers is to quickly find optimal solutions, a natural question arises: Are accurate models necessary to efficiently find such solutions? For the problems considered in this paper we have shown that BOA can still quickly find the optimal solution without an entirely accurate model. However, these models do not miss important dependencies between variables of the problem. Instead, they end up revealing a more complex structure than the original problem structure. Such fact has to do with model overfitting because the probabilistic models in EDAs are learned from the narrow subset of promising solutions (rather than the entire search space). While these models are efficient in guiding the search to optimal solutions, there is a need of having more accurate models when the knowledge of the problem structure is as important as

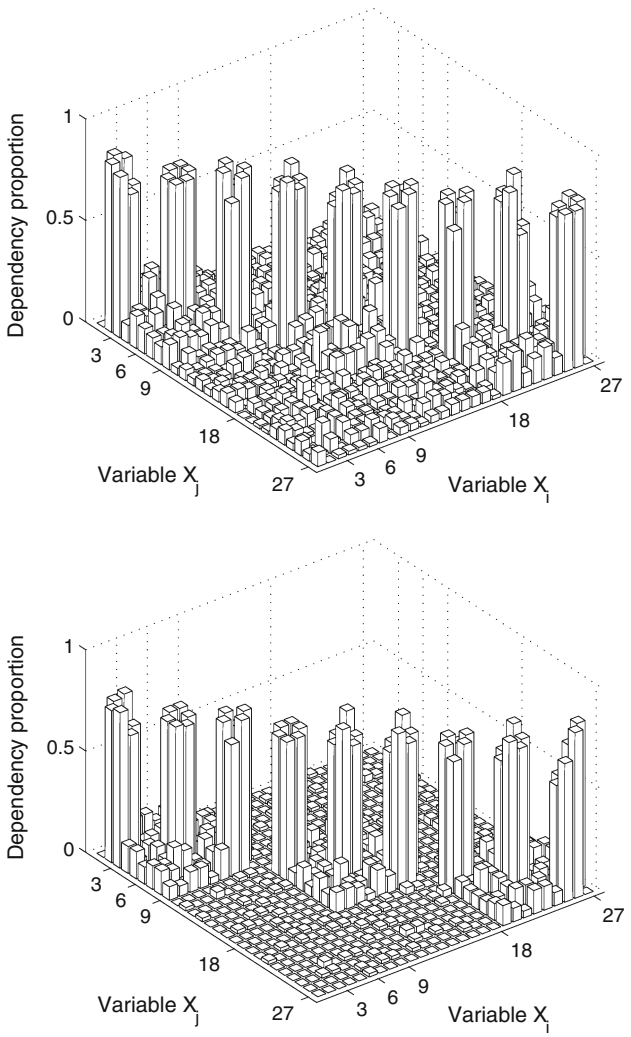


Fig. 17 Pairwise dependencies between variables for the 27-bit hierarchical trap problem, captured by hierarchical BOA in the last generation (when the optimum is found). The statistics presented do not consider the direction of the dependencies, which means that the graph is symmetrical with respect to the diagonal between points (0,0) and (27,27). The z-axis indicates the proportion of runs (out of 100 runs) in which a certain dependency was learned by the Bayesian network. For example, a dependency proportion of 0.87 between X_1 and X_2 means that in 87 out of 100 runs there was a dependency $X_1 \rightarrow X_2$ or $X_2 \rightarrow X_1$. The graph obtained with the $s-$penalty (*bottom*) is more informative about the underlying structure of the hierarchical problem than the standard metric (*top*)

achieving high-quality solutions. This is clearly the case when using model-based efficiency techniques or performing offline interpretation of the probabilistic models.

This paper makes a contribution towards understanding and improving model accuracy in the Bayesian optimization algorithm. Three main issues were addressed: first, a careful empirical analysis of model building in BOA was made to understand how the problem structure is captured by learned dependencies in the Bayesian networks, as well as when incorrect or unnecessary dependencies are introduced. Specifically, we have shown that the Bayesian–Dirichlet (K2 version) scoring metric produces more accurate models than the Bayesian information criterion metric, and that spurious dependencies are learned mainly at the end of the network construction. Additionally, we have identified the existence of $k-1$ different stages when learning strong interactions of order k, due to their different scoring metric contributions.

The role of selection in Bayesian network learning was also investigated by looking at selection as the mating pool distribution generator, which was found to have a great impact on model structural accuracy. Empirically, it has been shown that in BOA truncation selection produces considerably more accurate models than tournament selection. Intrigued by these results, we have made a theoretical analysis of both selection methods to understand the reason behind such quality difference. We have found that while tournament selection generates the mating pool according to a power distribution, which leads to model overfitting, truncation selection generates a more suitable uniform distribution for learning (with standard scoring metrics).

Finally, the metric gain originated from the resampling bias of tournament selection was modeled so that a quantitative measure could be obtained. Based on these results, we changed the complexity penalty used in the scoring metrics to counterbalance the corresponding metric gain in the same order of magnitude. The $s-$penalty which penalizes each edge/leaf addition by an additional factor of s (tournament size) was found to significantly improve the model structural accuracy. In essence, the greater the tournament size s is, the more demanding the metric should be in accepting edge/leaf additions. The proposed scoring metric was tested on the onemax, $m-k$ trap, and hierarchical trap problems, which present different challenges to probabilistic modeling in Bayesian EDAs. The results obtained for these problems show that the interpretability of the corresponding models has been significantly improved with respect to the standard penalty.

These results demonstrate that tournament selection is an efficient selection method for Bayesian EDAs, as it is for traditional genetic algorithms, as long as the scoring metric is adjusted according to its natural distribution in the mating pool. Additionally, truncation selection was found to be more appropriate when using standard scoring metrics. However, the corresponding model quality is still inferior to the one obtained for tournament with the $s-$penalty. In terms of function evaluations requirements, tournament selection is less expensive than truncation, particularly for lower values of selection intensity (typically used values). Although for the problems considered in this paper, tournament selection with the $s-$penalty has shown to be a better option than truncation with the

standard penalty, this might not be necessarily the case for other problems. The message to retain is that the inferior model accuracy obtained with the standard score metrics and tournament selection can be highly improved by simply using the $s-$penalty.

The study identifies both the selection operator and the scoring metric (that guides model search) as the main factors[1] to influence model quality in Bayesian EDAs. These results demystify previous empirical comparisons between different Bayesian EDAs, which mainly differ on the choice of the selection operator and scoring metric.

Overall, this work makes a step towards understanding and interpreting the probabilistic models in BOA, providing more interpretable models to assist efficiency enhancement techniques and human researchers. While we did not consider other selection operators such as ranking or proportionate selection, the methodology developed in the paper should provide enough guidelines to account for the non-uniform distributions generated by these operators.

Acknowledgments This work was sponsored by the Portuguese Foundation for Science and Technology under grants SFRH-BD-16980-2004 and PTDC-EIA-67776-2006, the BBSRC under grant BB/D0196131, the Air Force Office of Scientific Research, Air Force Materiel Command, USAF, under grant FA9550-06-1-0096, and the National Science Foundation under NSF CAREER grant ECS-0547013. Fernando Lobo is also a member of the Center for Environmental and Sustainability Research (CENSE) and acknowledges its support. The work was also supported by the High Performance Computing Collaboratory sponsored by Information Technology Services, the Research Award and the Research Board at the University of Missouri in St. Louis. The US Government is authorized to reproduce and distribute reprints for government purposes notwithstanding any copyright notation thereon.

References

Ackley DH (1987) A connectionist machine for genetic hill climbing. Kluwer Academic, Boston

Ahn CW, Ramakrishna RS (2008) On the scalability of the real-coded Bayesian optimization algorithm. IEEE Trans Evol Comput 12(3):307–322

Balakrishnan N, Nevzorov VB (2003) A primer on statistical distributions. Wiley

Blickle T, Thiele L (1997) A comparison of selection schemes used in genetic algorithms. Evol Comput 4(4):311–347

Brindle A (1981) Genetic algorithms for function optimization. PhD thesis, University of Alberta, Edmonton, Canada (unpublished doctoral dissertation)

Chickering DM, Geiger D, Heckerman D (1994) Learning Bayesian networks is NP-Hard. Technical Report MSR-TR-94-17, Microsoft Research, Redmond, WA

Chickering DM, Heckerman D, Meek C (1997) A Bayesian approach to learning Bayesian networks with local structure. Technical Report MSR-TR-97-07, Microsoft Research, Redmond, WA

Cooper GF, Herskovits EH (1992) A Bayesian method for the induction of probabilistic networks from data. Mach Learn 9:309–347

Cormen TH, Leiserson CE, Rivest RL (1990) Introduction to algorithms. MIT Press, Massachusetts

Correa ES, Shapiro JL (2006) Model complexity vs. performance in the Bayesian optimization algorithm. In: Runarsson TP et al (eds) PPSN IX: Parallel problem solving from nature, LNCS 4193, Springer, pp 998–1007

Deb K, Goldberg DE (1993) Analyzing deception in trap functions. Foundations of genetic algorithms 2, pp 93–108

Echegoyen C, Lozano JA, Santana R, Larrañaga P (2007) Exact bayesian network learning in estimation of distribution algorithms. In: Proceedings of the IEEE congress on evolutionary computation, IEEE Press, pp 1051–1058

Etxeberria R, Larrañaga P (1999) Global optimization using Bayesian networks. In: Rodriguez AAO et al (eds) Second symposium on artificial intelligence (CIMAF-99), Habana, Cuba, pp 332–339

Friedman N, Goldszmidt M (1999) Learning Bayesian networks with local structure. Graphical Models. MIT Press, pp 421–459

Goldberg DE, Sastry K (2010) Genetic algorithms: the design of innovation, 2nd edn. Springer

Goldberg DE, Korb B, Deb K (1989) Messy genetic algorithms: motivation, analysis, and first results. Complex Syst 3(5):493–530

Harik GR (1995) Finding multimodal solutions using restricted tournament selection. In: Proceedings of the sixth international conference on genetic algorithms pp 24–31

Harik GR, Lobo FG, Goldberg DE (1999) The compact genetic algorithm. IIEEE Trans Evol Comput 3(4):287–297

Hauschild M, Pelikan M (2008) Enhancing efficiency of hierarchical BOA via distance-based model restrictions. In: Proceedings of the 10th international conference on parallel problem solving from nature, Springer-Verlag, pp 417–427

Hauschild M, Pelikan M, Sastry K, Goldberg DE (2008) Using previous models to bias structural learning in the hierarchical BOA. In: Proceedings of the ACM SIGEVO genetic and evolutionary computation conference (GECCO-2008), ACM, New York, NY, USA, pp 415–422

Hauschild M, Pelikan M, Sastry K, Lima CF (2009) Analyzing probabilistic models in hierarchical BOA. IEEE Trans Evol Comput 13(6):1199–1217

Heckerman D, Geiger D, Chickering DM (1994) Learning Bayesian networks: the combination of knowledge and statistical data. Technical Report MSR-TR-94-09, Microsoft Research, Redmond, WA

Henrion M (1988) Propagation of uncertainty in Bayesian networks by logic sampling. In: Lemmer JF, Kanal LN (eds.) Uncertainty in artificial intelligence, Elsevier, pp 149–163

Japkowicz N, Stephen S (2002) The class imbalance problem: a systematic study. Intell Data Anal 6(5):429–450

Johnson A, Shapiro J (2001) The importance of selection mechanisms in distribution estimation algorithms. In: Proceedings of the 5th European conference on artificial evolution, LNCS vol 2310, Springer-Verlag, London, pp 91–103

Kubat M, Matwin S (1997) Addressing the curse of imbalanced training sets: one-sided selection. In: Proc. 14th international Conference on Machine Learning, Morgan Kaufmann, pp 179–186

Larrañaga P, Lozano JA (eds) (2002) Estimation of distribution algorithms: a new tool for evolutionary computation. Kluwer Academic Publishers, Boston, MA

Lima CF (2009) Substructural local search in discrete estimation of distribution algorithms. PhD thesis, University of Algarve, Portugal

[1] Given a sufficient population size (learning data set).

Lima CF, Sastry K, Goldberg DE, Lobo FG (2005) Combining competent crossover and mutation operators: a probabilistic model building approach. In: Beyer H et al (eds) Proceedings of the ACM SIGEVO genetic and evolutionary computation conference (GECCO-2005), ACM Press, pp 735–742

Lima CF, Pelikan M, Sastry K, Butz M, Goldberg DE, Lobo FG (2006) Substructural neighborhoods for local search in the Bayesian optimization algorithm. In: Runarsson TP et al (eds) PPSN IX: parallel problem solving from nature, LNCS 4193, Springer, pp 232–241

Lima CF, Goldberg DE, Pelikan M, Lobo FG, Sastry K, Hauschild M (2007) Influence of selection and replacement strategies on linkage learning in BOA. In: Tan KC et al (eds) IEEE Congress on evolutionary computation (CEC-2007), IEEE Press, pp 1083–1090

Lima CF, Pelikan M, Lobo FG, Goldberg DE (2009) Loopy substructural local search for the Bayesian optimization algorithm. In: Proceedings of the second international workshop on engineering stochastic local search algorithms (SLS-2009), LNCS Vol. 5752, Springer, pp 61–75

Lozano JA, Larrañaga P, Inza I, Bengoetxea E (eds) (2006) Towards a new evolutionary computation: advances on estimation of distribution algorithms. Springer, Berlin

Mühlenbein H (2008) Convergence of estimation of distribution algorithms for finite samples. (unpublished manuscript)

Mühlenbein H, Mahning T (1999) FDA—a scalable evolutionary algorithm for the optimization of additively decomposed functions. Evol Comput 7(4):353–376

Mühlenbein H, Schlierkamp-Voosen D (1993) Predictive models for the breeder genetic algorithm: I. Continuous parameter optimization. Evol Comput 1(1):25–49

Pearl J (1988) Probabilistic reasoning in intelligent systems: networks of plausible inference. Morgan Kaufmann, San Mateo, CA

Pelikan M (2005) Hierarchical Bayesian optimization algorithm: toward a new generation of evolutionary algorithms. Springer

Pelikan M, Goldberg DE (2001) Escaping hierarchical traps with competent genetic algorithms. In: Spector L, et al (eds) Proceedings of the genetic and evolutionary computation conference (GECCO-2001), Morgan Kaufmann, San Francisco, CA, pp 511–518

Pelikan M, Sastry K (2004) Fitness inheritance in the bayesian optimization algorithm. In: Deb K et al (eds) Proceedings of the genetic and evolutionary computation conference (GECCO-2004), Part II, LNCS 3103, Springer, pp 48–59

Pelikan M, Goldberg DE, Cantú-Paz E (1999) BOA: the Bayesian optimization algorithm. In: Banzhaf W et al (eds) Proceedings of the genetic and evolutionary computation conference GECCO-99, Morgan Kaufmann, San Francisco, CA, pp 525–532

Pelikan M, Goldberg DE, Lobo F (2002) A survey of optimization by building and using probabilistic models. Comput Optim Appl 21(1):5–20

Pelikan M, Sastry K, Goldberg DE (2003) Scalability of the Bayesian optimization algorithm. Int J Approx Reason 31(3):221–258

Pelikan M, Sastry K, Cantú-Paz E (eds) (2006) Scalable optimization via probabilistic modelling: from algorithms to applications. Springer

Pyle D (1999) Data preparation for data mining. Morgan Kaufmann, San Francisco, CA

Rissanen JJ (1978) Modelling by shortest data description. Automatica 14:465–471

Santana R, Larrañaga P, Lozano JA (2005) Interactions and dependencies in estimation of distribution algorithms. In: Proceedings of the IEEE congress on evolutionary computation, IEEE Press, pp 1418–1425

Santana R, Larrañaga P, Lozano JA (2008) Protein folding in simplified models with estimation of distribution algorithms. IEEE Trans Evol Comput 12(4):418–438

Sastry K (2001) Evaluation-relaxation schemes for genetic and evolutionary algorithms. Master's thesis, University of Illinois at Urbana-Champaign, Urbana, IL

Sastry K, Goldberg DE (2004) Designing competent mutation operators via probabilistic model building of neighborhoods. In: Deb K et al (eds) Proceedings of the genetic and evolutionary computation conference (GECCO-2004), Part II, LNCS 3103, Springer, pp 114–125

Sastry K, Pelikan M, Goldberg DE (2004) Efficiency enhancement of genetic algorithms via building-block-wise fitness estimation. In: Proceedings of the IEEE international conference on evolutionary computation, pp 720–727

Sastry K, Abbass HA, Goldberg DE, Johnson DD (2005) Sub-structural niching in estimation distribution algorithms. In: Beyer H, et al (eds) Proceedings of the ACM SIGEVO genetic and evolutionary computation conference (GECCO-2005), ACM Press

Sastry K, Lima CF, Goldberg DE (2006) Evaluation relaxation using substructural information and linear estimation. In: Keijzer M et al (eds) Proceedings of the ACM SIGEVO genetic and evolutionary computation conference (GECCO-2006), ACM Press, pp 419–426

Schwarz G (1978) Estimating the dimension of a model. Ann Stat 6:461–464

Thierens D (1999) Scalability problems of simple genetic algorithms. Evol Comput 7(1):45–68

Thierens D, Goldberg DE (1993) Mixing in genetic algorithms. In: Forrest S (ed) Proceedings of the Fifth international conference on genetic algorithms, Morgan Kaufmann, San Mateo, CA, pp 38–45

Weiss GM, Provost F (2003) Learning when training data are costly: the effect of class distribution on tree induction. J Artif Intell Res 19:315–354

Wu H, Shapiro JL (2006) Does overfitting affect performance in estimation of distribution algorithms. In: Keijzer M et al (eds) Proceedings of the ACM SIGEVO genetic and evolutionary computation conference (GECCO-2006), ACM Press, pp 433–434

Yu TL, Goldberg DE (2004) Dependency structure matrix analysis: Offline utility of the dependency structure matrix genetic algorithm. In: Deb K et al (eds) Proceedings of the genetic and evolutionary computation conference (GECCO-2004), Part II, LNCS 3103, Springer, pp 355–366

Yu TL, Sastry K, Goldberg DE (2007a) Population size to go: Online adaptation using noise and substructural measurements. In: Lobo FG, et al (eds) Parameter setting in evolutionary algorithms, Springer, pp 205–224

Yu TL, Sastry K, Goldberg DE, Pelikan M (2007b) Population sizing for entropy-based model building in genetic algorithms. In: Thierens D, et al (eds) Proceedings of the ACM SIGEVO genetic and evolutionary computation conference (GECCO-2007), ACM Press, pp 601–608

Yu TL, Goldberg DE, Sastry K, Lima CF, Pelikan M (2009) Dependency structure matrix, genetic algorithms, and effective recombination. Evol Comput 17(4):595–626